JOBSHEET 09

SERVER SIDE RENDERING (SSR)

PEMROGRAMAN BERBASIS FRAMEWORK

REIKA AMALIA SYAHPUTRI

2341720173

TI-3E / 19

TEKNOLOGI INFORMASI

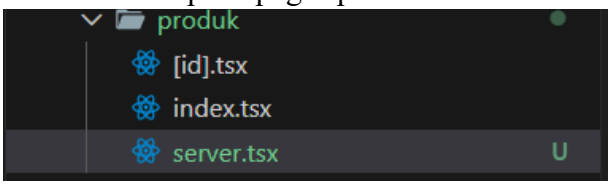
Github:

https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_09

C. Langkah Praktikum

Bagian 1 – Setup Halaman SSR

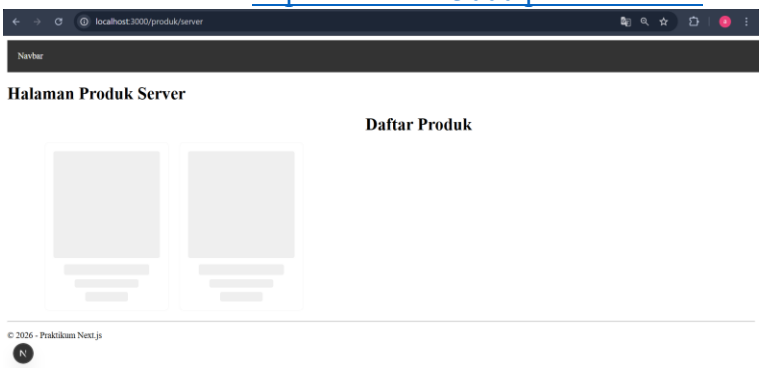
1. Buat file baru pada pages/products/server.tsx



2. Modifikasi file server.tsx :

```
import TampilanProduk from "../../views/produk";

const halamanProdukServer = () => {
  return (
    <div>
      <h1>Halaman Produk Server</h1>
      <TampilanProduk products={[]} isLoading/>
    </div>
  );
};

export default halamanProdukServer;
```

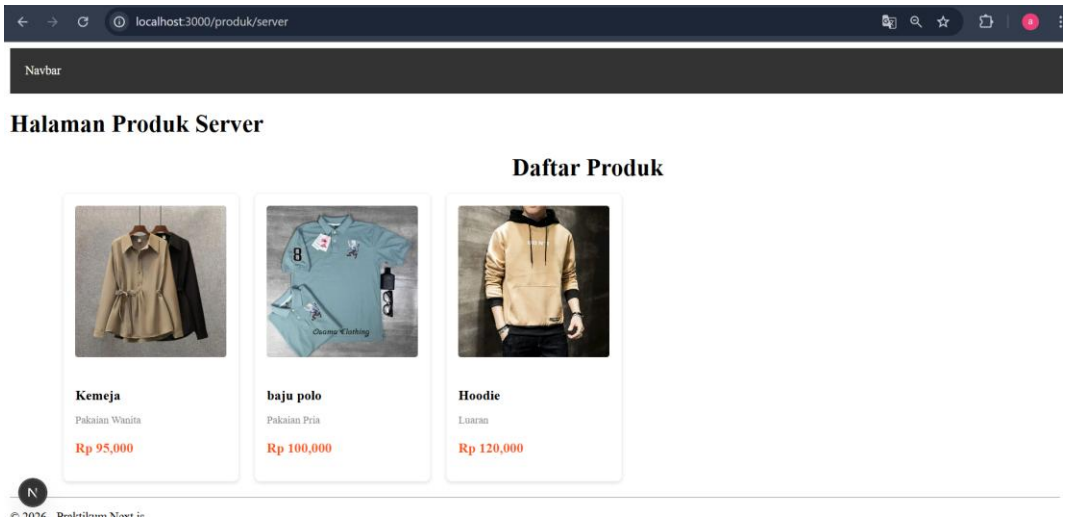
3. Jalankan browser : http://localhost:3000/produk/server



Bagian 2 – Implementasi getServerSideProps pada server.tsx

```typescript
import TampilanProduk from "../../views/produk";

type ProductType = {
  id: string;
  name: string;
  price: number;
  image: string;
  category: string;
};

const halamanProdukServer = (props: { products: ProductType[] }) => {
  const { products } = props;
  return (
    <div>
      <h1>Halaman Produk Server</h1>
      <TampilanProduk products={products} />
    </div>
  );
};

export default halamanProdukServer;

// Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
export async function getServerSideProps() {
  const res = await fetch("http://localhost:3000/api/produk");
  const respone = await res.json();

  // console.log("Data produk yang diambil dari API:", respone);

  return {
    props: {
      products: respone.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
    },
  };
}
```

- Jalankan browser http://localhost:3000/produk/server

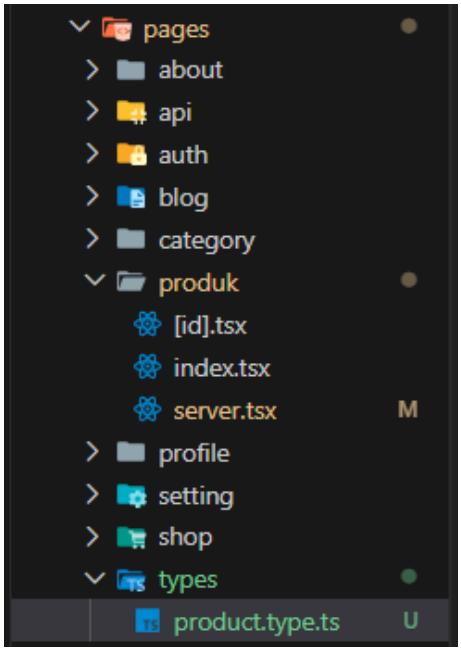


Catatan:

- Jika dijalankan maka skeleton tidak muncul yang akan muncul langsung pada produknya
- Harus menggunakan full URL
- Dipanggil setiap request halaman

Bagian 3 – Refactor Type (produk type)

1. Buat folder types pada folder pages dan buat file Product.type.ts



2. Modifikasi Product.type.ts

```
export type ProductType = {
    id: string;
    name: string;
    price: number;
    image: string;
    category: string;
}
```

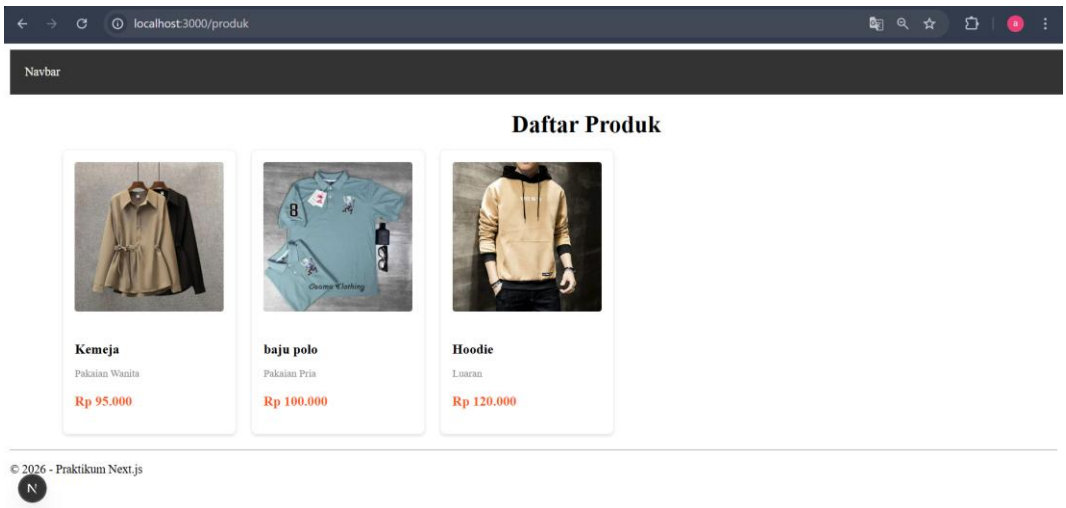
3. Setelah membuat file Product.type.ts maka modifikasi pada file server.tsx menjadi

```typescript
import TampilanProduk from "../../views/produk";
import { ProductType } from "../types/product.type";

// type ProductType = {
//   id: string;
//   name: string;
//   price: number;
//   image: string;
//   category: string;
// };


const halamanProdukServer = (props: { products: ProductType[] }) => {
  const { products } = props;
  return (
    <div>
      <h1>Halaman Produk Server</h1>
      <TampilanProduk products={products} isLoading={false} />
    </div>
  );
};

export default halamanProdukServer;

// Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
export async function getServerSideProps() {
  const res = await fetch("http://localhost:3000/api/produk");
  const respone = await res.json();

  // console.log("Data produk yang diambil dari API:", respone);

  return {
    props: {
      products: respone.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
    },
  };
}
```

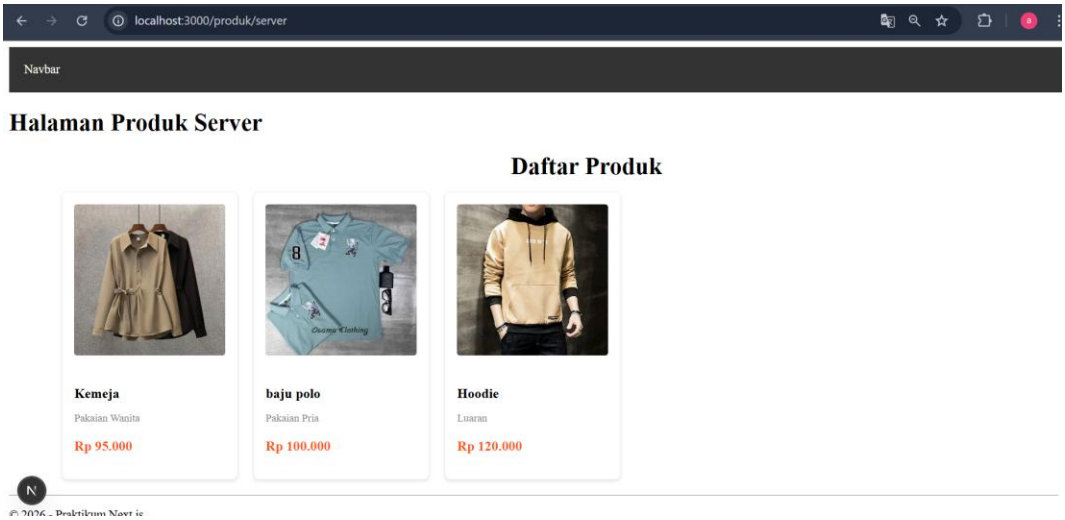
Bagian 4 – Uji Perbedaan SSR vs CSR

Uji 1 – Skeleton

- Buka halaman CSR

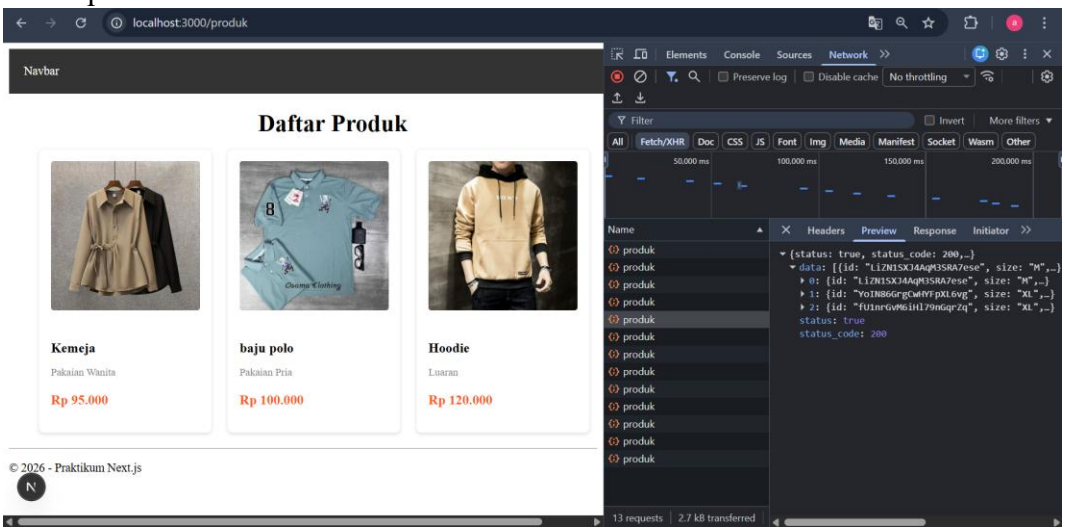


- Refresh
- Skeleton muncul
  Skeleton muncul setiap refresh halaman CSR namun dengan cepat akan digantikan dengan tampilan produk.
- Buka halaman SSR
- Refresh
- Skeleton tidak muncul

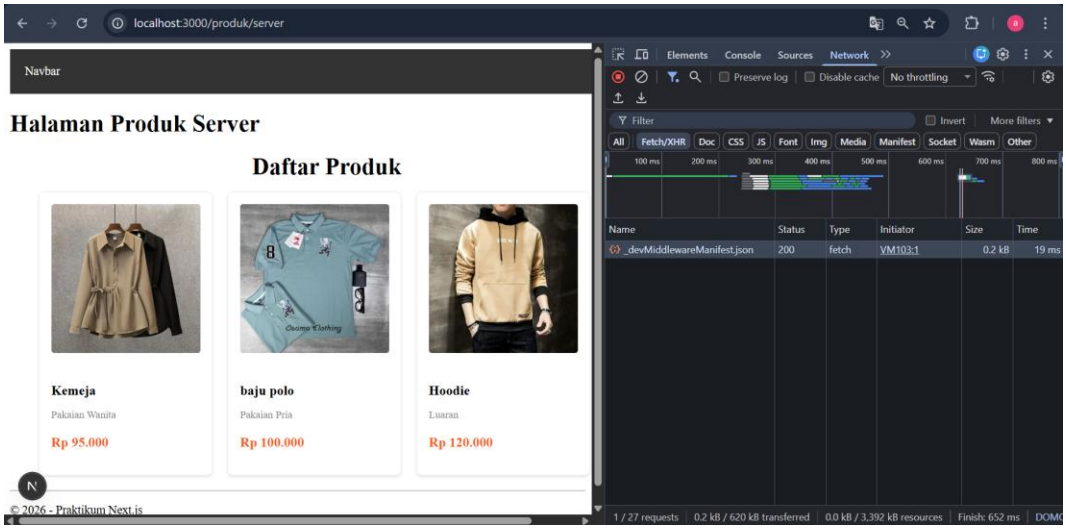


Uji 2 – Network Tab

1. Buka DevTools → Network → XHR
2. Refresh halaman CSR
   → Request API terlihat



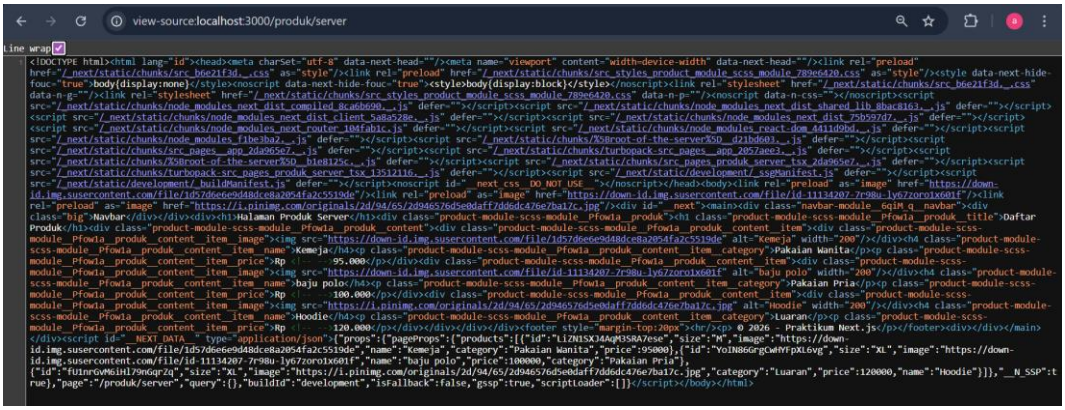
3. Refresh halaman SSR
   → Request API tidak terlihat



Uji 3 – Response HTML

- CSR: HTML awal kosong (berisi skeleton)

- SSR: HTML sudah berisi data produk lengkap



D. Tugas Praktikum

Tugas Individu

1. Buat 2 halaman:

- /products (CSR)

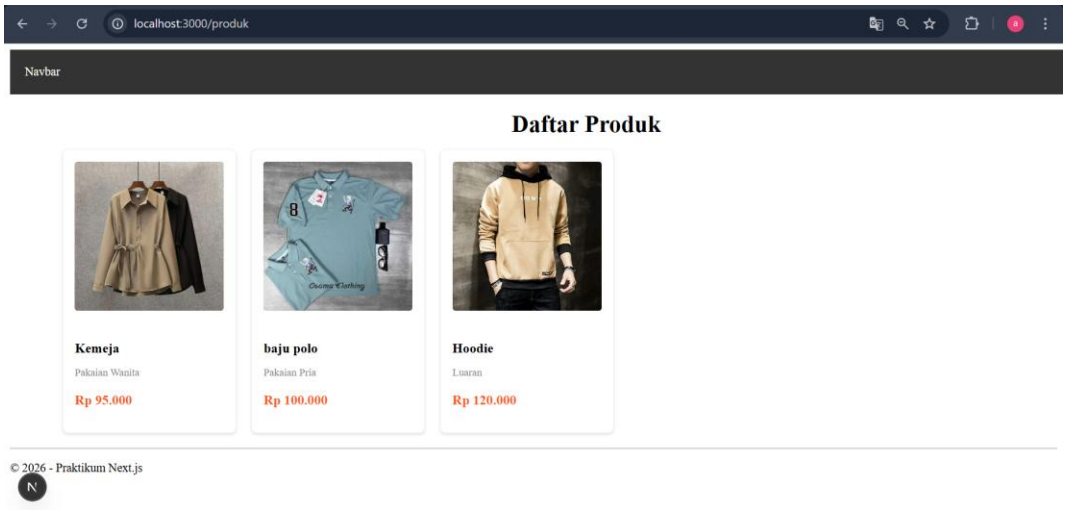
```python
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "@/views/produk";
import useSWR from "swr";
import fetcher from "../utils/swr/fetcher";

const kategori = () => {
    const [products, setProducts] = useState([]);
    const { data, error, isLoading } = useSWR("/api/produk", fetcher);

    return (
        <div>
            <TampilanProduk
            products={data?.data || []}
            isLoading={isLoading}
            />
        </div>
    );
};

export default kategori;
```

- /products/server (SSR)

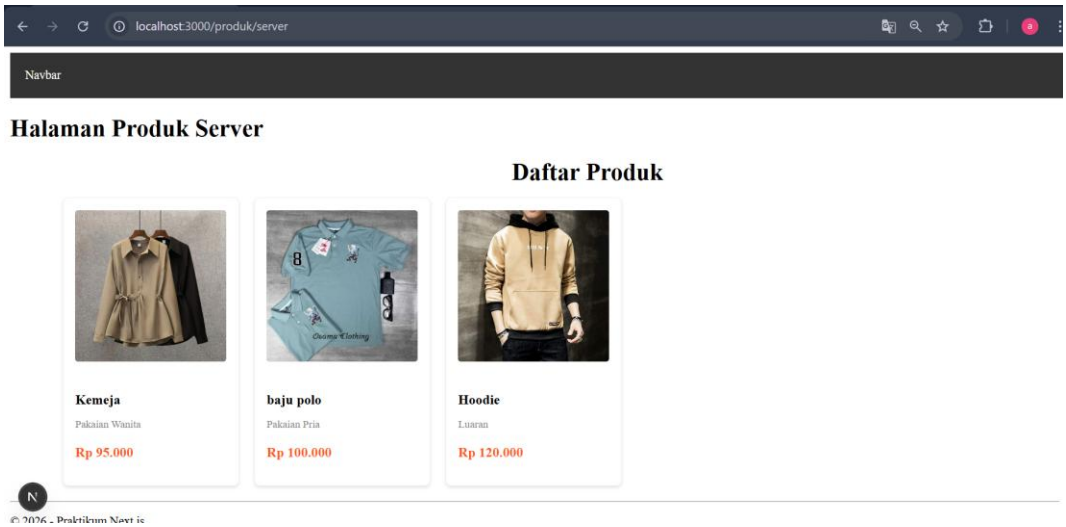
```python
import TampilanProduk from "../../views/produk";
import { ProductType } from "../types/product.type";

const halamanProdukServer = (props: { products: ProductType[] }) => {
  const { products } = props;
  return (
    <div>
      <h1>Halaman Produk Server</h1>
      <TampilanProduk products={products} isLoading={false} />
    </div>
  );
};

export default halamanProdukServer;

// Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan mengambil data produk dari API sebelum merender halaman.
export async function getServerSideProps() {
  const res = await fetch("http://localhost:3000/api/produk");
  const respone = await res.json();

  // console.log("Data produk yang diambil dari API:", respone);

  return {
    props: {
      products: respone.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
    },
  };
}
```

2. Dokumentasikan:

o Screenshot CSR

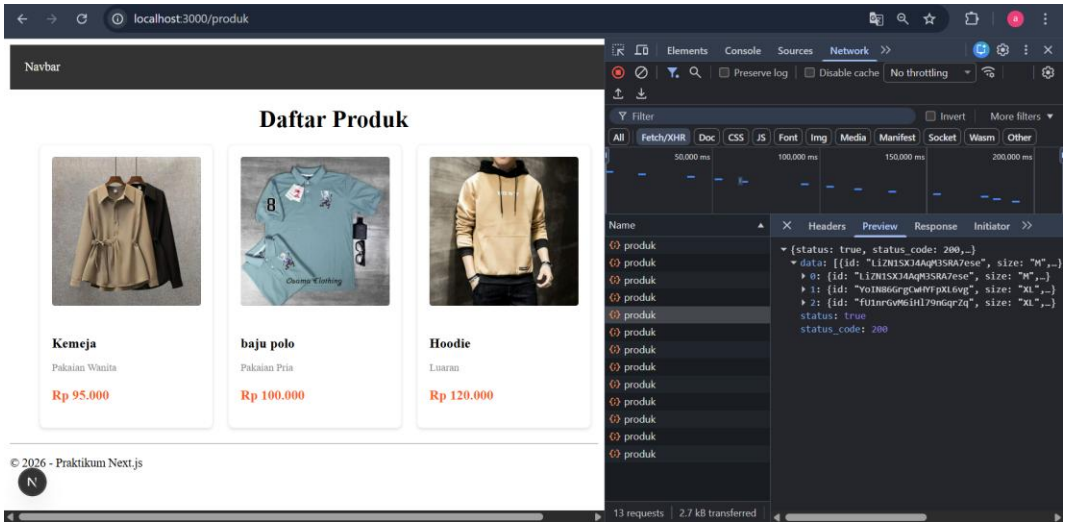


o Screenshot SSR

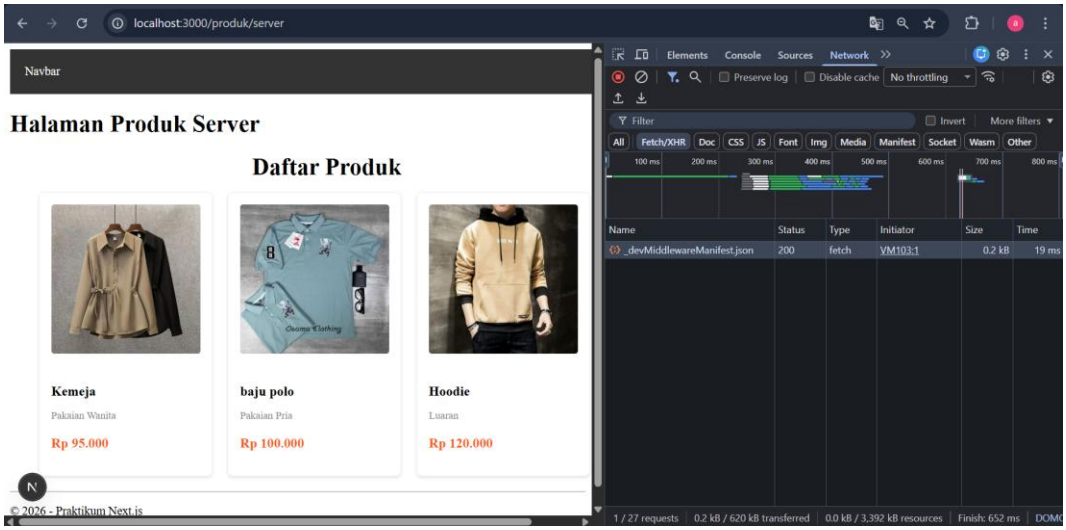


o Perbedaan Network tab

- CSR



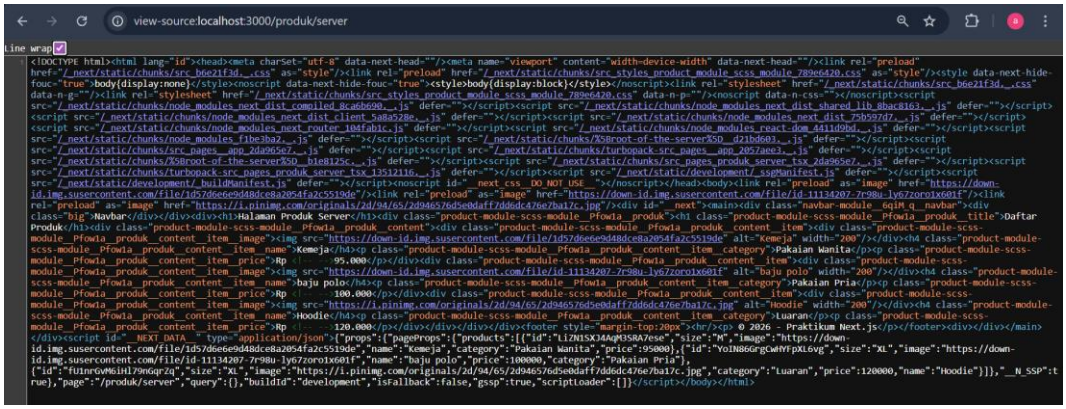
- SSR



o Perbedaan View Source

- CSR

- SSR



3. Buat laporan analisis minimal 2 halaman.

E. Studi Analisis

1. Mengapa SSR lebih baik untuk SEO?

- SSR (Server Side Rendering) lebih baik untuk SEO karena halaman sudah dirender lengkap di server sebelum dikirim ke browser. Artinya, ketika mesin pencari seperti Google melakukan crawling, konten halaman sudah tersedia dalam bentuk HTML penuh. Berbeda dengan CSR yang harus menunggu proses JavaScript dijalankan di browser, sehingga terkadang tidak semua konten langsung terbaca oleh mesin pencari.

2. Kapan sebaiknya menggunakan SSR?

- SSR sebaiknya digunakan ketika aplikasi membutuhkan data yang selalu terbaru setiap kali halaman diakses, misalnya pada halaman produk, berita, atau dashboard yang datanya sering berubah. Selain itu, SSR juga cocok digunakan jika aplikasi membutuhkan optimasi SEO agar lebih mudah ditemukan di mesin pencari.

3. Apa kekurangan SSR dibanding CSR?

- Kekurangan SSR dibanding CSR adalah proses rendering menjadi lebih berat di server karena setiap permintaan pengguna akan diproses dan dirender ulang di server. Hal ini bisa meningkatkan beban server dan memperlambat respon jika trafik tinggi. Selain itu, implementasi SSR biasanya lebih kompleks dibanding CSR.

4. Mengapa skeleton tidak muncul pada SSR?

- Skeleton tidak muncul pada SSR karena data sudah diambil dan dirender di server sebelum halaman dikirim ke browser. Saat halaman ditampilkan, data sudah tersedia sehingga tidak ada proses loading di sisi client. Skeleton biasanya digunakan pada CSR ketika data masih dalam proses pengambilan di browser.